

RTOS on the Raspberry Pi Pico

Hadi

Advanced Computer Systems 7507ENG

RTOS Laboratory Project

October 2026

Abstract—This report is about a small real-time operating system on a Raspberry Pi Pico. Several tasks share one processor. They show their work on an eight-digit seven-segment display and on a serial shell. The first scheduler gave every task the same 100 microsecond turn. We left that version running for at least five minutes. The later scheduler is preemptive, with turns of 1 second, 0.5 seconds, and 0.2 seconds. The shell is medium priority, so a high task cuts in and a low task waits. Tasks at the same priority share equal turns. When a higher task is removed, a lower one runs quicker, because nothing longer is still ahead of it. A task must loop forever. If it returns, the processor follows an empty link register. The report also explains a fault in `remove_Task`. After the start-up task removed itself, the display froze on HELLO and the shell never started.

Index Terms—RTOS, preemptive scheduling, round robin, Raspberry Pi Pico, testing

I. Introduction

The laboratory asks for a small operating system on a Raspberry Pi Pico. One processor has to share its time across several tasks. Each task is a function that loops and does not return. The tasks use the on-board LED, an eight-digit seven-segment display, and a USB serial shell.

The display board is wired to the Pico over SPI. Power and ground use the Pico supply pins. Chip select, clock, and data use pins 22, 24, and 25. A supplied hardware layer, running on the second core, copies an eight-byte buffer onto the display. Our tasks only write that buffer. Byte 0 is the left digit.

The work was done in two steps. The first scheduler gave every task the same short turn. This is round robin. We left that version running for at least five minutes. The second part asks for preemption from the scheduler, not from a new outside interrupt. A more important new task should be able to start at once. The save and load of the processor registers are supplied. Our code chooses which task runs next.

This report explains both schedulers, why a task must never return, how we checked them, and what went wrong when the start-up task removed itself.

II. The First Scheduler

The first RTOS was non-preemptive round robin. Every running task had the same turn. The priority number was saved on the task record, and the switch never read it.

The Pico clock is 125 MHz. `configure_Systick(1)` sets the reload value to $1 \times 125 - 1$, so SysTick fires every 1 microsecond. The handler adds one to Ticks. In this

first version it asked for a context switch whenever Ticks reached a multiple of 100. That is one turn every 100 microseconds, and the length was the same for every task.

scheduler did one job. It moved `CurrentTCB` to the next task in the linked list. If that pointer was empty, it went back to the first task, `Task_List`. `add_Task` put the new task at the end of the list and then returned. It did not ask for a switch. A new task waited until the current 100 microsecond turn ended.

No task could hold the processor, and no task could jump the queue. On the display, several digits move as if they are all running together. One hundred microseconds is too short to see a single turn. This version was meant to keep many tasks alive, not to show which one was more important.

A. The Five Minute Run

A short look can miss a fault that shows up later. After the shell was up we started several display tasks at once, on different digits, and left the Pico running for at least five minutes. This was the check of the equal-turn scheduler.

We watched three things. The digits must keep changing. The pattern must not freeze on HELLO or on one number. The shell must still answer. We pressed Enter, ran `pt`, and checked that the same tasks were still listed. At the end of the five minutes the digits were still moving and the shell still answered. The task list, the tick, and the display path stay alive when every task has the same short turn.

III. Why a Task Never Returns

A task is a function, and it must not finish. The laboratory asks for an endless loop so the function never returns. `count`, `flash`, `hexer`, `splat`, and the shell are all while (1) loops. `blink` flashes the LED the requested number of times, turns the LED off, and then sits in while (1). The job can end. The function cannot.

`add_Task` builds a fake saved stack, because a new task has never been switched out and has nothing real to restore. The program counter is the task function. The link register is left empty. On this stack that address is 0. Starting the task is like returning into the function. If the function then returns for real, the processor follows that empty link register and jumps to address 0.

The scheduler would still find the task. Ending the function does not take it off the list. Only `rt`, or `initd`

removing itself, does that. The next turn would try to resume a function that has already returned. The board can hang. The loop keeps the processor inside the task until someone removes it, or until reset.

The shell has a second reason to loop. If it returned, the prompt would be gone, and there would be no way to start or stop the other tasks. `initd` uses the same rule after it removes itself. It must spin until the next tick. Returning there would also follow the empty link register. That path is in Section VI.

IV. Why the Later Scheduler Works This Way

A. Preemption

The tick is still 1 microsecond. A more important task should not have to wait out someone else's whole turn, so each task now has its own limit. The task record holds quantum and `ticks_left`. Each tick subtracts one from `ticks_left`. At zero, the handler refills the limit and asks for a context switch. The scheduler still walks the list in order for that kind of switch.

The cut-in is separate, and it happens when a task is added. If the new priority number is higher than the running task, `add_Task` stores the new task in `PreemptTarget` and asks for a switch. On that switch the scheduler runs `PreemptTarget` first and then clears it. A new task with the same priority, or a lower one, is only linked onto the list. It waits.

The shell is started at medium priority. A high task typed at the shell cuts in. A low task typed at the shell waits. Tasks still take turns after that. Priority sets the length of a turn, and it lets a higher new task start immediately. It does not keep the processor on the high task forever.

B. Why Three Levels

The laboratory leaves the meaning of the priority number to us. We used three levels. A normal operating system groups work into a few bands, such as high, medium, and low. We used the numbers 3, 2, and 1 for those bands. Table I lists them.

TABLE I
The three priority levels

Level	Number	Time on each turn
High	3	1.0 s
Medium	2	0.5 s
Low	1	0.2 s

Another method is to let the priority number set the time. Priority 4 would run for four units, and priority 9 would run for nine units. Every different number would be its own speed. That is harder to explain, and it is harder to recognise on a small display. It also fits this program badly. `main` starts `initd` with priority 99. If the time followed the number, that one task would hold the processor for a very long turn.

With three fixed bands, the user tasks we care about are only low, medium, and high. Any other number is given the low time. Three bands look like a real system. A time that grows with the raw number does not.

C. Why the Shell Is Medium

The shell used to be priority 99. That was fine in the first scheduler. The switch never read the priority, so every task got the same 100 microsecond turn. The number 99 did nothing.

Only 1, 2, and 3 are real levels. Any other number, including 99, is given the low time of 0.2 seconds. A bigger number does not get a longer turn. Once the scheduler uses priority, 99 is not its own level. The shell would fall through to the low time.

The shell is medium, priority 2. A high task is above it, so that task starts at once. A low task is below it, so that task waits. We can see both on the display.

It cannot be high. High is priority 3, the top real level. A new task cuts in only when its number is higher than the shell. Nothing we type is higher than 3, so a high task would wait too. No task would jump in.

It cannot be low. Low is priority 1. Medium and high are both above that, so both would jump in at once. We would never see a task wait.

D. Why 1 Second, 0.5 Seconds, and 0.2 Seconds

The three limits are 1000000, 500000, and 200000 ticks. At 1 microsecond per tick, that is 1 second, 0.5 seconds, and 0.2 seconds.

We first used 100, 50, and 20 ticks. The ratio is the same, but those turns last 100 microseconds, 50 microseconds, and 20 microseconds. The display tasks step a digit every 0.25 seconds or slower. A turn that short is over before the eye can see who had the processor. The display looks as if every task is moving at once, so the levels cannot be told apart.

We kept the same ratio and stretched it until a person can watch it. One second, half a second, and 0.2 seconds are far enough apart. A high task holds a digit for about a second. A low task gets a short burst of about 0.2 seconds. These times are for the test. A finished product would use much shorter turns. Here the goal was to see the schedule on the screen.

V. Testing the Preemptive Scheduler

We started the display tasks and left them running on the display, then changed priorities and removed tasks.

A. Tasks at the Same Priority

Two tasks with the same priority number do not cut in on each other. The new one is not higher than the one already running, so it joins the list and waits. Both get a turn of the same length. Two high tasks each hold the processor for about a second. Two low tasks each get the short burst of about 0.2 seconds.

On the screen the two digits take turns and move at the same kind of pace. One does not race ahead of the other. The steps inside count, flash, hexer, and splat are themselves a large fraction of a second, so those equal turns are easy to see.

B. What Happens After a Higher Task Is Removed

We also ran a higher task and a lower task on different digits at the same time. The higher digit kept updating for its longer turn. The lower digit updated in a shorter burst, then waited while the higher task and the shell took their turns.

We then removed the higher task with `rt` and watched the digit that was left. The lower task moved on, and it ran quicker. Its turn came around sooner, because the long turn that had been in front of it was gone. With no higher task still ahead of it, the lower task took that place. The scheduler kept coming back to it, and its digit stepped more often than it had while the higher task was still in the list.

The priority number on the lower task did not change. It looked like the higher one because the list was shorter and nothing with a longer turn was left in front of it.

VI. The Fault When `initd` Removed Itself

A. What We Saw

The laboratory says the first task, `initd`, must start the hardware layer, write `HELLO`, start the shell, and then remove itself. We added that last step:

```
add_Task(&cmdShell, 0,
        "cmdShell", PRIO_MED);
my_id = CurrentTCB->id;
remove_Task(my_id);
while (1) {}
```

The display showed `HELLO` and then froze. The serial port never printed shell `#`.

We commented out the two lines that read the current id and call `remove_Task`. The shell came back. `count` and `flash` could then draw over `HELLO`. `HELLO` was fine. The shell task was fine. Those two lines in `initd` are what the laboratory asks for. The fault was inside `remove_Task`, and it showed up when a task removed itself.

B. Why It Happened

`remove_Task` only takes the task off the list. It does not switch. The processor stays inside `initd` until the next tick, and that task's stack lives inside its own record. `CurrentTCB` has to keep naming `initd` until the save has finished.

The old code unlinked `initd`, then pointed `CurrentTCB` at `cmdShell` while `initd` was still running. The next save wrote `initd`'s registers over the shell's new stack and wiped the frame that was supposed to start the shell. The shell never started. `HELLO` stayed on the display because the other core keeps drawing it.

The function did not move `CurrentTCB`, but it always freed the record. That frees the stack the processor is still

using. Freeing is safe when the shell removes some other task. It is not safe when `initd` removes itself.

C. The Fix

We made this change on the non-preemptive scheduler, before preemption was added. We left `CurrentTCB` alone. We free the record only when it is not the task that is running. The running task stays allocated until reset. A missing id still returns 0.

```
if (prev == NULL)
    Task_List = curr->next;
else
    prev->next = curr->next;

int is_current = (CurrentTCB == curr);
ENABLE_INT();

if (!is_current)
    free(curr);

return (int)Task_ID;
```

VII. Conclusion

The first scheduler gave every task the same 100 microsecond turn and ignored priority. We left that version on for at least five minutes. The digits kept moving and the shell still answered. Every task loops forever, because a return would follow the empty link register planted when the task was created.

The later scheduler uses three bands. High, medium, and low get 1 second, 0.5 seconds, and 0.2 seconds. The shell is medium, so a high task cuts in and a low task waits. Tasks at the same priority share equal turns. After the higher task is removed, the lower one runs quicker and takes that place, because nothing longer is still ahead of it.

A fault in `remove_Task` had pointed `CurrentTCB` at the shell while `initd` was still running. The next save overwrote the shell's start frame, `HELLO` froze, and the shell never printed. Leaving `CurrentTCB` alone, and not freeing the running task, fixes that path. After the fix, `HELLO` appears, the shell starts, and `initd` is gone from the list.

Acknowledgment

The context switch, the display hardware layer, and the helpers `readln` and `delay` came with the laboratory template. The scheduler, the task list, the tick handler, the shell, and the five tasks are our code.